

Day 59: Manual Testing and Debugging

Day 59: Manual Testing and Debugging

1. Day Number

Day 59

2. Topic Name

Basic Testing and Debugging

Today you will learn how to check whether a small Python program behaves correctly before learning automated testing tools such as pytest.

3. Connection

You have already built many small programs using:

- functions
- classes and objects
- conditions
- loops
- file handling
- bank account operations

Until now, you may have mostly run a program once and checked whether it worked.

Today you will learn a better habit:

Run the program with different inputs and compare the actual output with the expected output.

This is called **manual testing**.

4. Important Topics

Test cases

A **test case** is one situation that you intentionally test.

For example:

Password entered: correct password Expected result: Login successful

Another test case:

Password entered: wrong password Expected result: Access denied
--

Expected output

Before running your program, think:

| What should happen for this input?

Example:

```
Starting balance = 1000
Deposit = 500

Expected balance = 1500
```

Then run the program and compare.

```
Expected: 1500
Actual:   1500
```

If they match, the test passes.

Print debugging

Sometimes your program gives the wrong result, but you do not know why.

Temporary `print()` statements can help you inspect values.

Example:

```
balance = 1000
amount = 200

print("Balance before:", balance)

balance = balance + amount

print("Balance after:", balance)
```

These debugging prints help you see what happens step by step.

Edge cases

An **edge case** is an unusual input that can expose a problem.

For a bank program:

```
deposit = -100
withdraw = 0
withdraw = 5000 when balance is only 1000
wrong password
```

Testing only normal situations is not enough.

5. Foundational Notes

What is testing?

Testing means checking whether your program behaves as expected.

Think of this simple pattern:

```
Input
↓
Program
↓
Actual Output
↓
Compare with Expected Output
↓
Pass / Fail
```

Normal case vs edge case

A normal case could be:

```
Balance = 1000
Deposit = 500
```

An edge case could be:

```
Balance = 1000
Deposit = -500
```

Good programmers test **both**.

What is debugging?

Testing and debugging are related but different.

Testing finds that something is wrong.

```
Expected balance: 1500
Actual balance:    500
```

Debugging investigates why it is wrong.

For example:

```
print("balance =", balance)
print("amount =", amount)
```

You might discover that your code accidentally subtracts the deposit.

A useful testing habit

For every important operation, think about three kinds of cases:

Normal input
Invalid input
Boundary / edge input

For example, for withdrawal:

Normal:
withdraw 200 from balance 1000

Invalid:
withdraw -200

Edge:
withdraw 1000 from balance 1000

6. Easy Example

Imagine this very small function:

```
def add_money(balance, amount):  
    return balance + amount
```

You could manually test it like this:

Test 1

Starting balance: 100
Amount: 50
Expected result: 150

Then run it and check the result.

Another test:

Test 2

Starting balance: 0
Amount: 100
Expected result: 100

The important idea is not the function itself.

The important idea is:

```
Choose input  
↓  
Predict output  
↓  
Run program  
↓  
Compare result
```

7. Problem Statement

Create a **simple manual test checklist** for a bank account program.

The bank account program should be tested for:

1. Correct password
2. Wrong password
3. Deposit
4. Withdraw
5. Insufficient balance

You do **not** need to build a complicated testing system.

For each test, identify:

Input
Expected result
Actual result
Pass or Fail

Bank Account Manual Test Checklist

- ☐ **Correct password**
 - Enter the correct password.
 - Expected: Access should be allowed.
- ☐ **Wrong password**
 - Enter an incorrect password.
 - Expected: Access should be denied.
- ☐ **Valid deposit**
 - Start with a known balance and deposit a positive amount.
 - Expected: The balance should increase by the deposit amount.
- ☐ **Valid withdrawal**
 - Withdraw a positive amount smaller than or equal to the available balance.
 - Expected: The balance should decrease by the withdrawal amount.
- ☐ **Insufficient balance**
 - Try to withdraw more money than the available balance.
 - Expected: The withdrawal should be rejected and the balance should remain unchanged.
- ☐ **Negative deposit**
 - Try to deposit a negative amount.
 - Expected: The deposit should be rejected.
- ☐ **Negative withdrawal**
 - Try to withdraw a negative amount.
 - Expected: The withdrawal should be rejected.

8. Concepts Used

For today's exercise, you may use:

- variables
- `input()`
- `print()`
- `if`
- `elif`
- `else`
- comparison operators
- functions or methods
- object attributes if using OOP
- expected output
- actual output
- test cases
- edge cases
- debugging with `print()`

You are **not** learning automated unit-testing libraries yet.

9. Thought Process

Suppose your bank account starts with:

```
Password = python123  
Balance = 1000
```

Step 1: Test authentication

First ask:

```
What should happen with the correct password?
```

Expected:

```
Access allowed
```

Then test:

```
What should happen with a wrong password?
```

Expected:

```
Access denied
```

Step 2: Test deposit

Suppose:

```
Balance = 1000  
Deposit = 500
```

Calculate the expected result manually:

```
1000 + 500 = 1500
```

So:

```
Expected balance = 1500
```

Run your program and compare.

Step 3: Test withdrawal

Suppose:

```
Balance = 1500  
Withdraw = 300
```

Expected:

```
1500 - 300 = 1200
```

Step 4: Test insufficient balance

Suppose:

```
Balance = 1200  
Withdraw = 2000
```

The program should **not** allow it.

Expected:

```
Insufficient balance  
Balance remains 1200
```

Step 5: Test invalid amounts

Now deliberately enter:

```
Deposit = -100
```

or:

```
Withdraw = -500
```

Ask:

| Should a bank account accept this operation?

No.

Your validation should prevent the balance from changing.

10. Pseudocode-Style Testing Steps

This is testing pseudocode, not the full Python solution.

```
SET correct password
SET starting balance

TEST 1: Correct password
    ENTER correct password
    EXPECT access allowed
    COMPARE actual result with expected result

TEST 2: Wrong password
    ENTER wrong password
    EXPECT access denied
    COMPARE actual result with expected result

TEST 3: Deposit
    NOTE current balance
    ENTER positive deposit amount
    CALCULATE expected new balance
    RUN deposit
    COMPARE actual balance with expected balance

TEST 4: Withdraw
    NOTE current balance
    ENTER valid withdrawal amount
    CALCULATE expected new balance
    RUN withdrawal
    COMPARE actual balance with expected balance

TEST 5: Insufficient balance
    ENTER amount greater than current balance
    EXPECT withdrawal rejected
    EXPECT balance unchanged

TEST 6: Negative amount
    ENTER negative deposit or withdrawal
    EXPECT operation rejected
    EXPECT balance unchanged
```

11. Suggested Solving Approach

Both approaches are suitable today.

Functional approach

You could organize logic into functions such as:

```
check_password(...)
deposit(...)
withdraw(...)
```

This is easier if you want to focus mainly on **testing concepts**.

OOP-based approach

Since you have already worked with BankAccount, you could continue with something conceptually like:

```
BankAccount
    holder
    balance

    deposit()
    withdraw()
    get_balance()
```

Then manually test each method.

For your current learning sequence, **OOP-based is a good choice** because it also reinforces the BankAccount work from earlier days.

12. Easy Edge Cases

Wrong password

```
Expected:
Access denied
```

Make sure no banking operation is performed.

Negative amount

Example:

```
Deposit = -500
```

Expected:

```
Invalid amount
Balance unchanged
```

Similarly:

```
Withdraw = -200
```

should not increase the balance accidentally.

Withdrawal greater than balance

Example:

```
Balance = 1000
Withdraw = 1500
```

Expected:

```
Insufficient balance
Balance = 1000
```

The balance should remain unchanged.

Extra useful case: zero amount

Try:

```
Deposit = 0
```

and:

```
Withdraw = 0
```

Think about whether your program should accept them.

For a simple program, treating them as invalid is reasonable because the amount should usually be **greater than 0**.

13. Expected Input Examples

Assume:

```
Correct password: python123
Starting balance: 1000
```

Example 1 — correct password

```
Password: python123
```

Example 2 — wrong password

```
Password: abc123
```

Example 3 — deposit

```
Deposit amount: 500
```

Example 4 — withdrawal

Withdraw amount: 300

Example 5 — insufficient balance

Starting balance: 1000

Withdraw amount: 1500

Example 6 — invalid amount

Deposit amount: -200

14. Expected Output Examples

Correct password

Access granted

Wrong password

Access denied

Successful deposit

If:

Balance = 1000

Deposit = 500

Expected:

Deposit successful

Balance: 1500

Successful withdrawal

If:

Balance = 1500

Withdraw = 300

Expected:

Withdrawal successful

Balance: 1200

Insufficient balance

If:

```
Balance = 1000
Withdraw = 1500
```

Expected:

```
Insufficient balance
Balance: 1000
```

Negative amount

```
Invalid amount
Balance unchanged
```

The exact wording does not matter. What matters is that the **behavior and balance are correct**.

15. Hint Only

Start with a known state:

```
password = "python123"
balance = 1000
```

Before running each test, write down:

```
Input:
Expected output:
Expected balance:
```

Then run your program and compare:

```
Expected balance: ?
Actual balance:   ?
```

If something is wrong, temporarily add debugging statements around the suspicious operation:

```
print("Before:", ...)
print("Amount:", ...)
print("After:", ...)
```

Do not immediately change the code when a test fails. First identify **which value became wrong and at which step**. This habit will make debugging much easier as your Python programs become larger.